Блоковая область видимости

- Введение
- Области видимости Javascript
- Глобальный объект
- Область видимости var и let
- Блок кода
- Область видимости with

Введение

В JS есть несколько концепций, связанных с **областью видимости** (*scope*), которые не всегда легко объяснимы начинающим разработчикам. Впрочем, я не уверен, что это надо объяснять начинающим. Мы прошли уже несколько тем, мы умеем выполнять **навигацию** по DOM, умеем **формировать** и **выводить** в браузер довольно сложные структуры данных, умеем организовывать **взаимодействие** с пользователем посредством стандартных окон. И все это мы делали без знаний области видимости.

И я не хочу сказать, что эти знания не нужны. Я хочу сказать – ...

> ***Важно***. Я хочу сказать – **Всему свое время!** (как говорил Заратустра ;)*

Время пришло. Кто хочет стать Middle (а именно к этой цели я стремлюсь на своих курсах) должен ориентироваться в таких понятиях как:

- область видимости
- область видимости функции / блока
- лексическая область видимости
- область имён
- глобальные / локальные переменные
- замыкание

- this
- приватные и публичные области

> ***Примечание***. Все представленные примеры демонстрируют понятие **области видимости** и не предназначены для разбора функций.
>
> ***Важно*.** Если из-за наличия функций материал урока вам покажется слишком сложным, урок можно пропустить, до изучения урока **Пользовательские функции**.

> ***Примечание***. К области видимости мы будем обращаться еще не раз по мере изучения других тем. Так, и в **функциях**, и в **объектах**, понятие области видимости играет чрезвычайно важную роль.

Области видимости JavaScript

В JS область видимости – это **текущий контекст в коде**. Область видимости может быть определена **локально** или **глобально**.

Давайте разбираться, где переменные и функции доступны, как менять контекст в коде и писать более быстрый и поддерживаемый код (который и отлаживать быстрее).

Глобальная область видимости

Не написав ни строчки кода, мы уже находимся в глобальной области видимости. Если переменная объявлена **вне функций** или **фигурных скобок {}**, то считается, что она определена **в глобальной области видимости**.

Иногда вместо термина **область видимости** используют термин **пространство имён**.

```
// переменная глобальной области видимости
```

```
var name = 'Denis';
```

Как только происходит объявление глобальной переменной, эту переменную можно использовать в **любом месте кода**, в том числе в **функциях**.

При работе с глобальными переменными существует вероятность пересечения имен, когда двум или более переменным присваивают **одинаковое имя**. В одной области видимости не должно быть переменных с одинаковыми именами.

Если объявлять переменные через **var**, то вторая переменная после объявления **перепишет первую**. Такое поведение нежелательно, т.к. код усложняется в отладке.

```
// объявляем глобальную переменную
var login = 'master';
// объявляем глобальную племенную еще раз
var login = true; // могли не заметить, что переменная уже используется
console.log(login); // true
```

Если переменные объявляются через **const** или **let**, то каждый раз, когда будет происходить пересечение имён, будет показываться **сообщение об ошибке**. Такое поведение тоже нежелательно.

```
// объявляем глобальную переменную
let login = 'master';
// объявляем глобальную переменную еще раз
let login = false; // Identifier 'login' has already been declared
```

> ***Важно***. Вследствие такого поведения, объявлять переменные глобальной области видимости **не рекомендуется**.

Несмотря на сказанное, использование переменных глобальной области видимости обычная практика в обучающих курсах.

Во-первых, программировать надо, а функции и объекты это, как правило, далеко не первые темы курса.

Во-вторых, писать дополнительные программные конструкции ради простых примеров - только загромождать демонстрационный код.

> ***Примечание***. Заполните **форму согласия** следующего утверждения.
>
> ☑ Я, _____, отдаю себе полный отчет в опасности использования глобальных переменных. Всю ответственность за возможные последствия применения переменных глобальной области видимости беру на себя.
>
> Дата _____. Подпись _____.

Локальная область видимости

Переменные, которые используются только в определенной части кода, считаются помещенными в локальную область видимости. Такие переменные называются **локальными**.

В JavaScript выделяют **следующие типы локальных областей видимости**:

- **область видимости блока**
- **область видимости функции**
- **лексическая область видимости**

Обычно у нас есть одна **глобальная область видимости**, и каждая определяемая функция несёт в себе локальную область видимости.

Все локальные области видимости создаются только в функциональных областях видимости, они не создаются циклами типа **for** или **while** или директивами типа **if** или **switch**.

Новая функция – новая область видимости.

Область видимости блока

Переменная, объявленная внутри фигурных скобок **{}** через **const** или **let**, доступна только **внутри фигурных скобок**.

Блочная область видимости является частным случаем области видимости функции, т.к. функции объявляются с фигурными скобками (кроме случаев использования стрелочных функций с неявным возвращением значения).

Область видимости блока рассмотрим далее, в разделе **Блок кода**.

Область видимости функции

Переменная, объявленная внутри функции, доступна только внутри функции. Код снаружи функции не имеет к ней доступа.

Если я определю функцию и задам внутри переменные, они принадлежат локальной области видимости.

```
// глобальная область видимости
var myFunction = function () {
      // локальная область видимости
};
```

Все переменные из **локальной области видимости не видны в глобальной области видимости**. К ним нельзя получить доступ снаружи напрямую. Пример:

example_1. Переменные уровня функции локальны

```
<script>
    var myFunction = function () {
        // локальная переменная
        var login = 'big-johni';
        console.log(login);
    };
    // вызов функции
```

```
    myFunction();  // big-johni
    console.log(login);  // ReferenceError: login is not defined
</script>
```

Переменная **login** относится к **локальной области видимости**, она не видна снаружи и поэтому не определена.

Функции не имеют доступа к областям видимости других функций**, когда они объявляются раздельно**, даже если одна функция используется в другой.

В примере ниже функция fnSecond не имеет доступа к переменной **first** .

example_2. Каждая функция создает свою область видимости

```
<script>
    // определение первой функции
    function fnFirst () {
        const first = `I'm variable of fnFirst`;
        console.log(first);
    }
    // определение второй функции
    function fnSecond () {
        // вызов первой функции
        fnFirst();
        // пробуем получить доступ к переменной first
        console.log(first); // Uncaught ReferenceError: first is not
        defined
    }
    // вызов второй функции
    fnSecond();
</script>
```

С другой стороны, каждая функция, **определённая внутри другой функции**, имеет свою локальную область видимости, связанную с **областью видимости внешней функции**.

С таким поведением функции связана лексическая область видимости и понятие - замыкание.

Лексическая область видимости

Если одна функция определена внутри другой, внутренняя имеет доступ к области видимости внешней. Это называется **лексической** областью видимости, или **замыканием**, или ещё **статической** областью видимости.

> ***Важно***. Лексическая область видимости это когда переменные из области видимости внешней функции доступны в области видимости вложенной функции.

В тоже время внешняя функция не имеет доступа к переменным внутренней функции.

example_3. Лексическая область видимости

```
<script>

    var outerFunction = function () {

        // внешняя переменная

        let outer = 'John';

        var innerFunction = function () {

            // внутренняя переменная

            let inner = 'Max';

            console.log('Значение внешней переменной: ' + outer); //
            My name is John

        };

        // Uncaught ReferenceError: inner is not defined
```

```
        console.log('Значение внутренней переменной: ' + inner);
        // вызов внутренней функции
        innerFunction();
    };
    // вызов внешней функции
    outerFunction();
</script>
```

С лексической областью видимости довольно просто работать – всё, что определено в область видимости родителя, доступно в области видимости потомка.

```
// область видимости A
var myFunction = function () {
     // область видимости A + B
     var mySecondFunction = function () {
          // область видимости A + B + C
     };
};
```

example_4. Цепочка вложенных функций создает разные области видимости

```
<script>
    var outerFunction = function () {
        // внешняя переменная
        let first = 'First';
        var firstInnerFunction = function () {
            // внутренняя переменная
            let second = 'Second';
            console.log("Вложенная функция firstInnerFunction:");
            console.log('First variable: ' + first);
```

```
            var secondInnerFunction = function () {
                console.log("Вложенная функция
                secondInnerFunction:");
                console.log('First variable: ' + first);
                console.log('Second variable: ' + second);
            }
            // вызов второй внутренней функции
            secondInnerFunction();
        };
        // вызов первой внутренней функции
        firstInnerFunction();
    };
    // вызов внешней функции
    outerFunction();
</script>
```

Подробнее замыкания разберем в теме, посвященной пользовательским функциям.

Область видимости достаточно объемная тема. Невозможно охватить сразу все вопросы. В темах **функций** и **объектов** пойдем дальше.

Глобальный объект

Глобальный объект предоставляет переменные и функции, доступные **в любом месте программы**. По умолчанию это те, что встроены в язык или среду исполнения. В браузере глобальный объект называется **window**.

> ***Примечание***. Объект **window** представляет собой окно, содержащее **DOM документ**.

Ко всем свойствам глобального объекта можно обращаться **напрямую**.

example_5. Глобальный объект window

```
<script>
    alert("Привет"); // это то же самое, что и
    window.alert("Привет");
    console.log(document.body); // body
    // это то же самое, что и
    console.log(window.document.body); // body
</script>
```

В браузере глобальные функции и переменные, объявленные с помощью **var**, **становятся свойствами глобального объекта**:

```
// объявляем глобальную переменную
var value = 5;
alert(window.value); // 5
```

При этом обычно рекомендуется использовать глобальные переменные как можно реже.

> ***К сведению***. Дизайн кода, при котором функция получает входные параметры и выдаёт определённый результат, чище, надёжнее и удобнее для тестирования, чем когда используются **внешние**, а тем более **глобальные переменные**.

Если вы начинающий программист, то подобная ремарка может быть непонятной. Но, как только мы пройдем **Пользовательские функции**, все станет на свои места.

Если свойство или несколько свойств настолько важные, что вы хотите сделать их доступными для всей программы, **запишите их в глобальный объект** и сделайте более специфичными, как продемонстрировано в примере **example_6**.

example_6. Пользовательские данные в глобальном объекте window

```
<script>
    // сделать информацию о пользователе можно глобальной,
    // и более специфичной
    window.myData = {
        login: "denis",
        level: "macho"
    };
    // доступ к переменной где угодно в коде
    alert(myData.login); // denis
    // получить login из window можно явно
    console.log(window.myData.login); // denis
</script>
```

Свойство name глобального объекта window

window.name

Возвращает строковое значение - **имя окна**. Свойство должно быть именно **string**. Всегда. При присвоении свойству **числа**, оно автоматически преобразуется в **строку**.

Поддерживается во всех браузерах.

```
window.name = "Имя_окна";
```

Примечание. Окнам не обязательно иметь имя.

Если присвоить **window.name** какое-то текстовое значение, например "test", и **перезагрузить страницу**, то при повторном обращении к **window.name**, вы получаете сохраненное свойство **name** со значением "test". Такое поведение свойства позволяет сохранять и использовать данные после перезагрузки страницы. Это могут быть любые строковые данные, в том числе в **JSON формате**.



Используя подобное поведение свойства **name** объекта **window** можно передавать данные между страницами.

example_7. Использование свойства name глобального объекта

```
<script>
    // присвоим свойству name строковые данные
    window.name = "I'll be back!";
</script>
```

В example_8 представлен код принимающей стороны.

example_8. Передача данных в свойстве name объекта window

```
<script>
    // тестируем свойство
    document.write("<h1>", window.name, "</h1>");
</script>
```

В одной из самостоятельных работ передадим между страницами **JSON-объект**.

> ***Примечание***. Передача данных между сценариями с помощью свойства **name** объекта **window** это "костыль". Но он работает и про него надо знать. Правильные способы обмена данными между сценариями пройдем позднее.

Область видимости var и let

Во втором уроке первой темы я написал следующее примечание:

> ***Примечание***. Тестирование различий между переменными **let** и **var**, будет проведено нами позже, пока эти различия не имеют для нас решающего значения.

Пришло время более подробно разобраться с этими ключевыми словами.

Оператор var

Оператор **var** объявляет одну или несколько переменных в текущей области видимости, **инициализируя** их, при необходимости.

Синтаксис

```
var varname1 [= value1 [, varname2 [, varname3 ... [, varnameN]]]];
```

Параметры

- **varnameN** – имя переменной, может использоваться любой допустимый идентификатор.
- **valueN** – начальное значение переменной, может быть любым допустимым выражением.

Описание

Объявление переменной всегда обрабатывается до выполнения кода, где бы она ни находилась. Область видимости переменной, объявленной через **var**, это её текущий **контекст выполнения**. Контекст выполнения может ограничиваться **функцией** или быть **глобальным**, для переменных, объявленных за пределами функции.

> ***Таким образом***. Область видимости переменной - **текущая функция**. Если переменная объявлена вне функции, область видимости - **глобальный объект window**.

Присвоение значения необъявленной переменной подразумевает, что она будет создана как **глобальная переменная**. Переменная становится свойством глобального объекта **window**.

example_9. Объявление переменных внутри функции

```
<script>
```

```
    // определение функции
    function fnTest() {
        y = 10; // y - глобальная
        var x = 20; // x - уровня функции
    }
    // вызов функции
    fnTest();
    // проверяем переменные
    console.log("y = ", y); // 10
    console.log("x = ", x); // Uncaught ReferenceError: x is not
    defined
</script>
```

Различия между **объявленной** и **необъявленной** переменными следующие:

- Объявленные переменные ограничены **контекстом выполнения**, в котором они были объявлены. Необъявленные переменные всегда **глобальны**.
- Объявленные переменные инициализируются **до выполнения любого кода**. Необъявленные переменные не существуют до тех пор, пока к ним не выполнено присваивание.
- Необъявленные переменные можно **удалять**.

example_10. Удаление необъявленных переменных

```
<script>
    var a = 1;
    b = 2;
    console.log("a = ", window.a); // 1
    console.log("b = ", window.b); // 2
    console.log("---------------");
    delete window.a; //
```

```
    delete window.b; // свойство 'b' было удалено и больше не
    существует
    console.log("a = ", window.a); // 1
    console.log("b = ", window.b); // undefined
</script>
```

Использовать **var** вне функций **не обязательно**. Вы можете объявить переменную, просто присвоив ей значение. Однако хорошим стилем будет использование **var**, который необходим в функциях, если имеется **глобальная переменная с тем же именем**.

Рассмотрим взаимодействие глобальных переменных с переменными, объявленными внутри функции.

example_11. Взаимодействие переменных

```
<script>
    // глобальные переменные
    var a = 5;
    var b = 20;
    function go() {
        // область видимости переменной - глобальный window
        a = 50;
        // область видимости переменной - текущая функция
        var b = 200;
    }
    go();
    // значение внешней (глобальной) переменной a поменялось
    console.log("a = ", a); // 50
    // значение внешней (глобальной) переменной b не поменялось
    console.log("b = ", b); // 20
</script>
```

Из-за перечисленных различий, использование необъявленных переменных может привести к непредсказуемым последствиям. **Рекомендовано всегда объявлять переменные, вне зависимости, находятся они внутри функции или в глобальном контексте.**

Оператор let

Оператор **let** объявляет переменную с **блочной областью видимости** с возможностью инициализировать её значением.

Синтаксис

```
let varname1 [= value1 [, varname2 [, varname3 ... [, varnameN]]]];
```

Параметры

- **varnameN** – имя переменной, может использовать любой допустимый идентификатор.
- **valueN** – начальное значение переменной, может быть любым допустимым выражением.

Описание

Оператор **let** позволяет объявить локальную переменную с **областью видимости, ограниченной текущим блоком кода**. В отличие от ключевого слова **var**, которое объявляет переменную глобально или локально во всей функции, независимо от области блока.

> ***Важно***. У объявлений переменной через **let** есть несколько основных отличия от **var**.

- **Область видимости переменной let – блок {...}.**

Переменная, объявленная через **var**, видна везде в функции. Переменная, объявленная через **let**, видна только в рамках блока **{...}**, в котором объявлена.

example_12. Let объявление переменных

```
<script>
    var x = 5; // x через var
    let y = 5; // y через let
    // блок кода
    {
        var x = 10;
        let y = 10;
    }
    console.log("x через var = ", x); // 10
    console.log("y через let = ", y); // 5
</script>
```

Это, в частности, влияет на объявления внутри **if**, **while** или **for**.

- **Переменная let видна только после объявления.**

Переменные **var существуют и до объявления**. Они равны **undefined**:

```
console.log(a); // undefined
var a = 5;
```

С переменными **let** всё проще. До объявления их вообще нет.

Такой доступ приведёт к ошибке:

```
console.log(a); // Uncaught ReferenceError: Cannot...
let a = 5;
```

Переменные **let** нельзя повторно объявлять. То есть, такой код выведет ошибку:

```
let x;
let x; // Uncaught SyntaxError: Identifier ...
```

- **let не создаёт свойства на глобальном объекте**

На верхнем уровне скриптов и функций **let**, в отличии от **var**, не создаёт свойства на **глобальном объекте**. Например:

example_13. Свойства глобального объекта

```
<script>
    var x = 'global x';
    let y = 'global y';
    // проверяем свойства глобального window
    console.log(window.x); // global x
    console.log(window.y); // undefined
</script>
```

В выводе программы будет отображена строка **global x** для window.x, но **undefined** для window.y.

- **При использовании в цикле, для каждой итерации создаётся своя переменная.**

Блок кода

Блок кода или **блок инструкций** используется для группировки нуля или более инструкций. Блок отделяется парой **фигурных скобок** и может опционально быть **поименован**.

Синтаксис

```
[name:]{
    statement_1;
    statement_2;
    ...
    statement_n;
```

}

Параметры

- **name** – необязательное имя для визуальной идентификации или использования как точки выхода для оператора **break**.
- **statement**_1, **statement**_2, ..., **statement**_n – вызовы внутри блока

Описание

Как самостоятельная конструкция используется редко. Как правило, используется с **операторами управления потоком** (if, for, while и т.п.).

```
var auto = "Lada";
if ( auto == "Rolls Roys")
{
      alert ("Жизнь удалась!");
} else
{
      alert ("Упс... :(");
}
```

> ***Примечание***. Хотя отдельные блоки и являются корректными с точки зрения языка, но в JavaScript их не используют или используют редко.

Область видимости блока с использованием var

Переменные, объявленные через **var**, **не имеют блочной области видимости**. Переменные, введённые внутри блока, имеют областью видимости содержащую их функцию или скрипт, и последствия записи в них значения распространяются за границы, собственно, блока, в котором они объявлены.

Другими словами, блок инструкций **не вводит новую область видимости**. Хотя блоки не являются нарушением синтаксиса, не стоит использовать их слишком часто, потому что они не делают то, чего вы от них ожидаете.

Пример самостоятельного использования блока:

```
var x = 1;
{
      var x = 2;
}
console.log(x); // 2
```

В консоль выводится 2, потому что оператор **var x внутри блока** - в той же области видимости, что и оператор **var x перед блоком**.

Область видимости блока с использованием let и const

Наоборот, идентификаторы, объявленные с помощью **let** и **const**, имеют **блочную область видимости**:

```
let x = 1;
{
     let x = 2;
}
console.log(x); // 1
```

x = 2 ограничено рамками области видимости блока, в котором оно было определено.

То же самое и для **const**:

```
const c = 10;
{
     const c = 2;
}
console.log(c); // 10
```

> ***Обратите внимание***. Объявленные внутри блока **let** и **const**

переменные не приводят к ошибке **SyntaxError: Identifier has already been declared** (идентификатор уже был объявлен), потому что переменная может быть объявлена внутри области видимости единожды.

Разбираемся дальше.

Область видимости with

Блок **with** добавляет новую область видимости.

Синтаксис

```
with(object)
    statement;
```

Параметры

- **object** – добавляет указанный объект в **цепочку областей видимости**. Может быть любое выражение, скобки вокруг обязательны.
- **statement** – **блок** или **JavaScript-вызов** для выполнения внутри дополненной области видимости.

Ниже представлен альтернативный синтаксис объявления блока **with** совместно с блоком кода.

```
with(object){
    statement_1;
    statement_2;
    ...
    statement_n;
}
```

В следующем примере вычисления выполняются с областью видимости, определенной объектом **Math**.

example_14. Определяем область видимости объекта Math

```
<script>
```

```
    var maxNum = 10;
    var minNum = 1;
    // получение случайного числа
    // используем функции Math.random(), Math.floor()
    with (Math) {
        let maxNum = 5;
        float = random() * (maxNum - minNum + 1) + minNum;
        num = floor(float) ;
    }
    // вывод расчетных значений
    console.log(float);
    console.log(num);
</script>
```

Важно. При поиске переменной внутри **with**, интерпретатор сначала проверяет свойства **object**. **Если свойства с таким именем нет - он идет дальше вверх по областям видимости**.

Пример использования блока **with** и коллекции HTML элементов представлен в example_15.

example_15. Область видимости with и коллекция HTML элементов

```
<script>
    with(document.querySelector("h2").style) {
        // избавились от необходимости каждый раз при упоминании
        свойства писать цепочку доступа к HTML-элементу
        color = "red";
        fontSize = "36pt";
        fontFamily = "Arial";
```

```
        fontWeight = "900";
    }
</script>
```

Использование **with** с одной стороны наглядно, с другой - может скрывать ошибки программирования, и соответствующий код плохо сжимается компрессорами. По этим причинам **злоупотреблять** оператором **with** не рекомендуется.

Вместо этого можно использовать чуть менее наглядный, но надежный доступ через **временную переменную**:

```
// создание временной переменной
el = document.querySelector("h2").style;
// доступ к свойствам через временную переменную
el.color = "red";
el.fontSize = "36pt";
el.fontFamily = "Arial";
el.fontWeight = "900";
```

Если все понятно, двигаемся дальше. А со следующим объектом будет все также легко и намного интереснее. Хотя, все-таки сложнее, немного. Но интереснее - точно.

Допустим, мы имеем следующий объект:

```
let data = {
    id_user : 3,
    user:{
        name : "Евгений",
        surname : "Даугава",
        login : "master",
        email : "pechora-pro@mail.ru"
    },
```

```
    personal:{
        inn : 123456789,
        police : "12-V23",
        certificate :"yes",
        phone : "8(912)552-92-25"
    }
};
```

Объект сложнее предыдущих примеров, но это ничто, по сравнению с возможными объектами реальных приложений, так что тренируемся.

Используя область видимости **with**, получаем доступ к **свойствам объекта data**:

```
with(data) {
    console.log(id_user);  // тест доступа
}
```

Получаем доступ к **свойствам объекта user**:

```
with(data.user) {
    console.log(name);  // тест доступа
}
```

Получаем доступ к **свойствам объектов data**, **user**:

```
// более гибкий способ работы с объектом data
with(data) {
    console.log(id_user);
    with(user){
        console.log(name);
    }
}
```

Тестируем сказанное на примере.

example_16. Вложенные области видимости

```
<script>
    let data = {
        id_user : 3,
        user:{
            name : "Евгений",
            surname : "Даугава",
            login : "master",
            email : "pechora-pro@mail.ru"
        },
        personal:{
            inn : 123456789,
            police : "12-V23",
            certificate :"yes",
            phone : "8(912)552-92-25"
        }
    };
    with(data) {
        with(user){
            var html = `
            Пользователь: ${name} ${surname} <p>
            Идентификатор в системе: ${id_user} <p>
            `;
        }
    }
    document.write(html);
</script>
```

Обратите внимание, не обнаружив идентификатора id_user в области видимости data.user интерпретатор будет подниматься **вверх по областям видимости** вплоть до глобальной в поисках указанной переменной.

В **самостоятельных работах** к уроку рассмотрим:

- Расчет выражений, имеющих скобочные группы.
- Чтение значений переменных, находящихся в цепочке областей видимости.
- Работа со свойством name глобального объекта window.
- Вывод объектов, используя области видимости with.

* - и да, Заратустра ничего подобного не говорил...